

Relatório de Engenharia Reversa: Formato de Preset Matribox II Pro

Este documento detalha a análise do binário `libapp.so` do software Matribox II Pro, focando na lógica de codificação e cifragem dos arquivos de preset (`.prst`).

1. Visão Geral do Formato

O arquivo `.prst` é estruturado como um array JSON de inteiros decimais, que representam os bytes do arquivo após um processo de ofuscação e codificação em Base64. A estrutura lógica segue o pipeline:

1. **Dados Brutos (JSON)** -> 2. **Cifragem Dinâmica (XOR/LCG)** -> 3. **Encapsulamento JSON (Array de Bytes)** -> 4. **Base64**.

2. O Algoritmo de Cifragem (LCG + Rolling XOR)

A análise identificou que o "miolo" dos dados de áudio é protegido por uma cifra de fluxo baseada em um **Linear Congruential Generator (LCG)**.

Constantes do LCG

O gerador utiliza as seguintes constantes determinísticas (padrão comum em implementações Dart/C++ para sistemas embarcados):

- **Multiplicador (A):** `1103515245` (`0x41C64E6D`)
- **Incremento (C):** `12345` (`0x3039`)
- **Módulo (M):** `2^31`

Fluxo de Inicialização (Seed)

1. O arquivo inicia com um cabeçalho estático seguido por **4 bytes de Semente (Seed)**.
2. Esta semente é gerada a partir do `timerMillisecondClock` no momento do salvamento.
3. O estado interno do LCG é inicializado com esta semente: `state = seed`.

Processo de Mascaramento

Para cada byte de dado limpo (`plainByte`), o algoritmo executa:

1. Atualização do estado: `state = (A * state + C) & 0x7FFFFFFF`.
2. Geração da chave: `key = (state >> 16) & 0xFF`.

3. Cifragem: `cipherByte = plainByte ^ key` .

3. Estrutura do Arquivo e Validação

Cabeçalho e Metadados

Os arquivos `.prst` contêm blocos identificados por IDs de 32 bits em **Little Endian**. Antes da cifragem, os dados são organizados em uma stream dinâmica.

Checksum

A integridade é validada por um **Checksum global** simples (soma de bytes ou CRC32 truncado) aplicado sobre a string JSON antes da conversão final para Base64. O binário faz referência a `HTModelPreset.decodefunc` para validar se o arquivo é compatível com o hardware ("Matribox II PRO").

4. Implementação em TypeScript (Pseudo-código)

TypeScript

```
class MatriboxCrypto {
  private state: number;
  private readonly A = 1103515245;
  private readonly C = 12345;
  private readonly M = 0x80000000;

  constructor(seed: number) {
    this.state = seed;
  }

  private nextKey(): number {
    this.state = (this.A * this.state + this.C) % this.M;
    return (this.state >> 16) & 0xFF;
  }

  public transform(data: Uint8Array): Uint8Array {
    const result = new Uint8Array(data.length);
    for (let i = 0; i < data.length; i++) {
      result[i] = data[i] ^ this.nextKey();
    }
    return result;
  }
}
```

// Exemplo de uso para exportação

```
function exportPreset(jsonData: string) {  
    const seed = Date.now() & 0xFFFFFFFF;  
    const crypto = new MatriboxCrypto(seed);  
    const plainBytes = new TextEncoder().encode(jsonData);  
    const encryptedBody = crypto.transform(plainBytes);  
  
    // O formato final envolve anexar a seed e converter para o array JSON espe  
    const finalArray = [ ...seedBytes, ...encryptedBody ];  
    return btoa(JSON.stringify(finalArray));  
}
```

5. Conclusão

A proteção não é uma criptografia forte (como AES), mas uma ofuscação dinâmica para evitar edições simples e garantir a integridade entre versões de firmware. O uso do LCG com a semente baseada em tempo explica por que o conteúdo muda a cada salvamento, mesmo sem alterações nos parâmetros do preset.